Milestone: 20GB Reached

We've reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original

GET SH*T DONE: Meta-prompting and Spec-driven Development for Claude Code and Codex

GSD ("Get Shit Done") aims to solve context rot, the quality degradation as the model's context window fills.

Agent Native Follow
~9 min read · February 23, 2026 (Updated: February 23, 2026) · Free: No

The fix is to stop treating a single chat thread as your build system.

GSD forces the work into **small, checkable plans** and runs each plan in a **fresh context window**, with **atomic git commits** per task so you can bisect your way out of agent chaos.

In this post, I'm going to unpack what GSD actually is, why it shows up *now*, and the parts I think are genuinely non-obvious, like the **"discuss phase"** step that locks in product decisions *before* planning, and the **Nyquist validation layer** that tries to make sure you have a test feedback loop *before you write the code*.

I'll also give you concrete patterns you can steal even if you never install GSD.

Context first, tools second

AI coding assistants write thousands of lines while you review, and you have to understand the failure modes.

You typically lose time to:

- **Long-session drift** where the agent slowly forgets constraints
- **Invisible decision loss** where you agreed on a pattern but agentstops following it
- **Unreviewable diffs** where agent makes one giant commit without narrative or rollback points

- **Fake progress** where agent writes lots of code but produces a little working product

Failure Mode	Root Cause	GSD Countermeasure
It worked earlier, now it's inventing stuff	Context rot / drift	Fresh context per plan; keep main session light
It built the wrong UI behavior	Unmade product decisions	Discuss-phase captures preferences into CONTEXT.md
The plan is too big; execution goes sideways	Tasks exceed what fits reliably in one context	2–3 atomic plans per phase; plan-check loop
We don't know what broke	Massive diff / no traceability	Atomic commits per task
It compiles but doesn't work	No feedback loop	Verify-work UAT + Nyquist validation mapping

Because of these reasons, GSD is built as the context engineering layer that makes Claude Code and other runtimes more reliable, by designing a workflow around fresh contexts, externalized state, and verification.

It supports Claude Code, OpenCode, Gemini CLI, and Codex, and it ships as an installer you run via `npx`.

It's also evolving fast and there are frequent releases, actively fighting real-world edge cases (health checks, auto-advance chaining, context window monitoring, plan verification gates, requirements traceability, etc.).

> *Hey, thanks for reading this article. I'm writing a deep-dive ebook on **Agentic SaaS**, the emerging design patterns that are quietly powering the most innovative startups of 2026.*
>
> ***You can grab the first chapter here:** [Agentic SaaS Patterns Winning in 2026](), packed with real-world examples, architectures, and workflows you won't find anywhere else.*

GSD in one sentence

GSD is a spec-driven development workflow + prompt/agent harness that tries to prevent context rot by externalizing state into files, splitting work into small plans, executing each plan in a fresh context, and verifying output against explicit goals.

It wraps AI coding tools with:

- **A consistent artifact set** (`PROJECT.md`, `REQUIREMENTS.md`, `ROADMAP.md`, `STATE.md`, per-phase plans, summaries, verification reports, etc.)
- **A phase-based workflow** (discuss → plan → execute → verify, repeat)
- **Multi-agent orchestration** (specialized agents for research, planning, execution, verification; thin orchestrator ties it together)
- **Explicit verification loops** (plan-checking iterations; manual UAT; debugging subagents)
- **Git hygiene as a first-class constraint** (atomic commits per task; traceability)

The core problem: context rot

Even when agents have large context windows, quality doesn't stay constant as context grows.

GSD's design response is brutally pragmatic:

1. **Externalize the system's memory into files.**

2. **Split work into small tasks that fit comfortably in a fresh context.**
3. **Attach verification to each task.**
4. **Make the output easy to audit and revert.**

If you are more interested in learning about memory system, read the following article:

OpenClaw Memory Systems That Don't Forget: QMD, Mem0, Cognee, Obsidian

If your agent has ever randomly ignored a decision you know you told it... it's not random.

medium.com

How GSD works

Here's the lifecycle as GSD documents it. I'll keep it concrete.

0) Install and pick a runtime

```
npx get-shit-done-cc@latest
```

The installer prompts for runtime (Claude Code, OpenCode, Gemini CLI, Codex, or all) and whether to install globally or locally.

GSD also strongly recommends running Claude Code with:

```
claude --dangerously-skip-permissions
```

...which is a *real* tradeoff we'll come back to.

1) Initialize: `/gsd:new-project`

After installation, you can run

```
/gsd:new-project
```

It asks questions until it understands the idea, optionally spawns researchers, extracts requirements, and produces a roadmap.

It creates core artifacts like `PROJECT.md`, `REQUIREMENTS.md`, `ROADMAP.md`, `STATE.md`, plus research output.

If you already have code, it suggests starting with:

- `/gsd:map-codebase` to analyze stack/architecture/conventions/concerns, so subsequent planning is grounded in how your repo actually works.

2) The non-obvious step: `/gsd:discuss-phase N`

GSD inserts a step that I think is *the entire point*:

Discuss the phase before researching or planning, to lock in preferences and decisions that the model would otherwise guess.

```
/gsd:discuss-phase 1
```

Copy

It explicitly calls out the kinds of gray areas that routinely cause misalignment:

- **Visual features**: layout density, interactions, empty states
- **APIs/CLIs**: response format, flags, error handling, verbosity
- **Content systems**: structure, tone, depth
- **Org tasks**: grouping criteria, naming, duplicates, exceptions (GitHub)

This produces a `{phase}-CONTEXT.md` file that feeds the researcher and planner.

3) Plan: `/gsd:plan-phase N`

```
/gsd:plan-phase 1
```

Copy

Planning has three explicit pieces:

1. Research (guided by your phase context)
2. Plan creation (2–3 atomic tasks)
3. Plan verification (loop until pass, up to a limit)

The goal is to produce plans that are small enough to execute in a fresh context window.

GSD uses **XML prompt formatting** for each plan, with explicit verification steps baked in.

```
<task type="auto">
  <name>Create login endpoint</name>
  <files>src/app/api/auth/login/route.ts</files>
  <action>
    Use jose for JWT (not jsonwebtoken - CommonJS issues).
    Validate credentials against users table.
    Return httpOnly cookie on success.
  </action>
  <verify>curl -X POST localhost:3000/api/auth/login returns 200 + Set-Cookie</verify>
  <done>Valid credentials return cookie, invalid return 401</done>
</task>
```

Copy

That structure is collapsing ambiguity.

4) Execute: `/gsd:execute-phase N`

Execution is where GSD leans hardest into fresh context or bust.

```
/gsd:execute-phase 1
```

Copy

It runs plans in dependency-based **waves**:

- Parallelize independent plans
- Sequence dependent plans
- Avoid file conflicts by adjusting grouping

And for each plan, it uses a **fresh context window** so the executor isn't polluted by whatever happened earlier.

Then it creates **atomic commits per task** which is an observability and rollback mechanism for AI automation.

```
PHASE EXECUTION

WAVE 1 (parallel)          WAVE 2 (parallel)          WAVE 3
 Plan 01    Plan 02   →    Plan 03    Plan 04   →    Plan 05
 User       Product        Orders     Cart           Checkout
 Model      Model          API        API            UI
      │          │              ↑          ↑              ↑
      └──────────┴──────────────┴──────────┘              │
            Dependencies: Plan 03 needs Plan 01           │
                          Plan 04 needs Plan 02           │
                          Plan 05 needs Plans 03 + 04     │
```

5) Verify: `/gsd:verify-work N`

This is manual UAT with structured help:

- Extract testable deliverables
- Walk you through each ("Can you log in with email?")
- If something fails, it spawns debug agents and generates fix plans for re-execution

```
/gsd:verify-work 1
```

This is exactly where vibe coding dies in production if not verified.

6) Milestones: audit and ship

GSD treats milestones as cycles:

- `/gsd:audit-milestone`
- `/gsd:complete-milestone` (archive + tag)
- `/gsd:new-milestone` (start the next version cycle) (GitHub)

This is a "release management" for solo developers, without pretending you need sprint theater.

Quick mode: `/gsd:quick`

For ad-hoc tasks, quick mode gives you the guarantees (state tracking, atomic commits) without the full research/plan-check/verifier loop.

How is this helpful in the real world?

It turns agent output into engineering artifacts

The most practical part of GSD is the file system.

It's generating *the scaffolding that makes code reviewable*:

- Project vision that stays loaded (`PROJECT.md`)
- Requirements with v1/v2 scope (`REQUIREMENTS.md`)
- Roadmap phases (`ROADMAP.md`)
- Decisions and blockers (`STATE.md`)
- Per-task plans + summaries + verification outputs

It constrains the blast radius

Atomic commits mean:

- `git bisect` can isolate the task that broke things
- reverts are surgical
- future agent sessions can reason about history more cleanly

This is a big deal because agent regressions are often non-local. The model changes five files to help, and you don't know which change mattered. Small commits are the antidote.

3) It makes parallelism real

GSD's wave model gives you a concrete way to parallelize tasks while respecting dependencies.

> *Vertical slices parallelize better than horizontal layers.*

That's just good engineering and it becomes critical when you have multiple executors writing code concurrently.

4) It forces you to decide

Most misalignment is because **you never specified the product decision** and the model guessed.

GSD makes that guess-work explicit, and gives you a place to pin decisions into a context file that later stages must read.

5) It introduces a quality gate before coding (Nyquist validation)

This is the spiciest idea and it's easy to miss.

During plan-phase research, GSD can map automated test coverage to each requirement *before* code is written, producing a `{phase}-VALIDATION.md` feedback contract.

The plan-checker treats missing verify commands as a failure condition.

Code and config

One thing I like is that GSD surfaces workflow tradeoffs as config.

The user guide describes `.planning/config.json` with toggles for research, plan-checking, verifier, Nyquist validation, and model profile selection.

Here's a simplified version you can paste as a starting point:

```
{
  "mode": "interactive",
  "depth": "standard",
  "model_profile": "balanced",
  "workflow": {
    "research": true,
    "plan_check": true,
    "verifier": true,
    "nyquist_validation": true
  },
  "planning": {
    "commit_docs": true,
    "search_gitignored": false
  },
  "git": {
    "branching_strategy": "none"
  }
}
```

And here's the **move fast, accept risk** shape:

```
{
  "mode": "yolo",
  "depth": "quick",
  "model_profile": "budget",
  "workflow": {
    "research": false,
    "plan_check": false,
    "verifier": false,
    "nyquist_validation": false
  },
  "planning": {
    "commit_docs": false
  }
}
```

Those exact fields and defaults are documented in the user guide, including a scenario table (prototyping vs production) and model-profile breakdown by agent.

Why spec + verify works

In classical software engineering, we get reliability from **tight feedback loops**:

- unit tests (fast feedback on logic)
- integration tests (feedback on boundaries)
- CI (repeatable truth machine)
- code review (human judgment)
- observability (production feedback)

Since agentic coding is just software engineering with a new failure mode (e.g., the coder is non-deterministic and sometimes overconfident), the stable principle is:

> ***The more autonomous the code generator, the more you must invest in verification, traceability, and rollback.***

GSD's Nyquist validation idea is basically this principle applied early, don't let the agent write code for requirements you can't verify quickly.

However there are few caveats.

You're generating and maintaining planning artifacts. That can be a feature or a tax. The config includes `commit_docs` and notes around `.gitignore` interactions for a reason.

Discuss-phase helps, but it doesn't eliminate ambiguity, you just moved it into a structured conversation. If you don't know what you want, you can't outsource that to a model, and if the agent can run shell commands and commit to your repo without friction, your security posture needs to match that reality.

Concluding thoughts

GSD has a few important contributions:

- **Context is a consumable resource.**
- **Memory belongs in files, not in a chat thread.**
- **Work should be sliced to fit into fresh contexts.**
- **Verification must be attached to tasks, not bolted on later.**
- **Git history is part of system observability.** (GitHub)

This makes you to think what kind of engineering org you are:

- Are you a developer chatting with a tool?
- Or are you a developer operating a build system where the code generator is non-deterministic?

GSD assumes the second, that's why it feels heavier than a prompt, and why it tends to feel more reliable than a raw chat.

Bonus Articles

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic...

medium.com

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you...

medium.com

OpenClaw Variants on $10 Hardware and 10MB RAM

[OpenClaw Variants on $10 Hardware and 10MB RAM Most product bugs show up when a simple feature lands on a box with a...]

medium.com

Observational Memory for Long-Running Agents — “This wasn’t the plan!”

Always-on agents have unbounded context growth problem.

medium.com

Fully Autonomous Companies: OpenClaw Gateway + Routing + Agents

Whether you think it’s hype or not, people are already trying to run fully autonomous companies on OpenClaw.

medium.com

Deep Research with 96,000+ Trajectories — Completely Offline

Imagine synthesizing human-like research trajectories exceeding 100 turns entirely offline, no reliance on search or...

medium.com

ClawRouter: Anthropic charged me $4,660 — How I cut it 70% with smart LLM routing

Last month I opened my credit-card statement and almost threw up. Anthropic charged me $4,660.87, just for AI APIs.

medium.com
#claude-code#gpt-5#openai-codex#gemini#agentic-ai

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description

Describe your problem here...

Submit Cancel